TASK 6 – Organise the Logic into Functions

Function design

rate_per_litre(fat, snf) accepts quality readings and returns the rate. payout(litres, fat, snf) returns the entry payment. ten_day_total(payments) accepts a sequence of payment amounts and returns their sum. validate_entry(litres, fat, snf) checks input constraints. Functions should return values so that callers can test, store, print or combine them; printing inside a calculation function makes reuse and automated testing harder.

Scope and docstrings

Names assigned inside a function are local by default. A function can read module-level constants without declaring them global. It should not reassign those constants. The global keyword is unnecessary and should be avoided for rate configuration. Every function should have a docstring explaining its purpose, parameters and return value.

Example implementation

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP = Decimal("0.1")

def rate_per_litre(fat, snf):
    """Return the rate per litre for the supplied fat and SNF readings."""
    fat_steps = ((fat - Decimal("3.5")) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - Decimal("8.5")) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    return (BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE).quantize(Decimal("0.01"))

def payout(litres, fat, snf):
    """Return the rounded payment for one delivery."""
    return (litres * rate_per_litre(fat, snf)).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def ten_day_total(payments):
    """Return the sum of payment amounts for a ten-day period."""
    return sum(payments, Decimal("0.00"))
```

Alternative and complexity

A single long script may initially contain fewer function calls, but calculation logic becomes duplicated and difficult to test. Lambdas are suitable for short expressions, not multi-step validation and financial calculations. For n entries, the overall calculation remains O(n) whether implemented with functions or inline statements. A function call adds a small constant overhead per call, so the total overhead is O(n), but it is normally insignificant compared with the benefits of reuse, clarity and testing. The functions use O(1) extra space per call, excluding stored payment data.